Evolving from AFAP – Teaching NICs about time

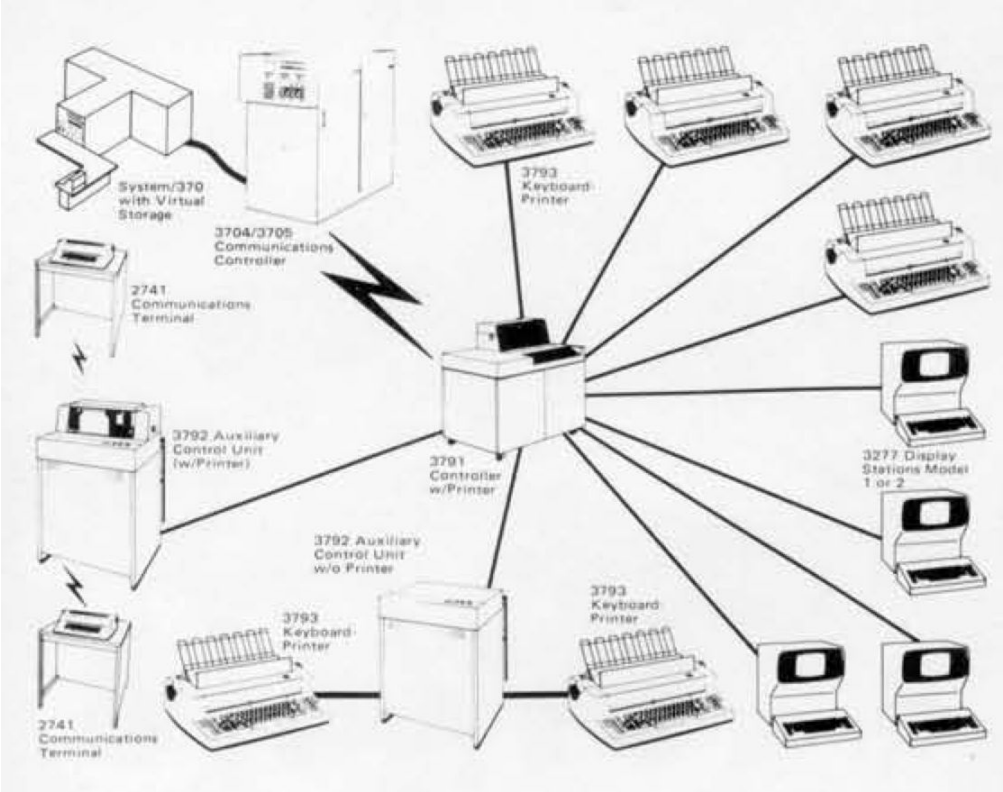
Van Jacobson



netdev 0x12, Montreal, July 2018

In the beginning ...

… ‘networking’ meant this:



IBM 3790 Controller (image from 1974 announcement brochure)

… because:

Computer vendors developed computer networks to:

a) lock-in customers
b) sell more hardware (remote access terminals & mainframes)
c) lock-in customers

… with the result that:

- CRT / printer / card reader engineers drove design
  - → device artifacts freely mixed with communication artifacts
- communication links were slow (300bps, 2.4Kbps, 56Kbps)
  - → huge premium on minimizing headers & implicit communication
- encapsulation expressed the mainframe-to-device controller path
  - → had to know path details to build a packet

IP/TCP was different

Major architectural emphasis on simplicity, expressive abstractions & implementable contracts.

IP interface-to-interface small messages; unreliable, best-effort delivery

TCP process-to-process byte streams; reliable, eventual delivery

These layers are notable for what they *don't* say – there's nothing about intended application, protocol efficiency or network structure.

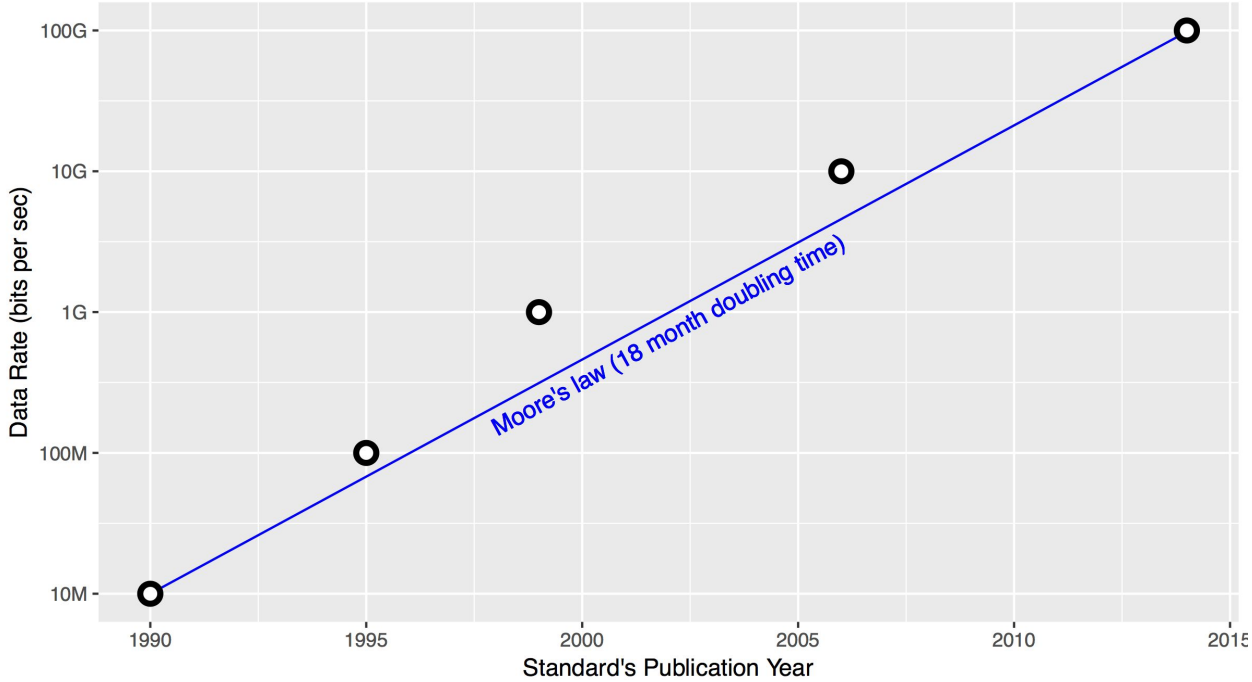
AFAP?

TCP's reliable delivery constrains *how much* is sent but not *how fast*.

- lead to an AFAP (as fast as possible)* output contract, usually implemented as device output queue(s).
  - → '*How fast*' is implicit in queue drain rate.
- receive window or queue length limit determines inflight between protocol & device.
- rate constraint is local & upstream of the wire.

*a.k.a. *work conserving*

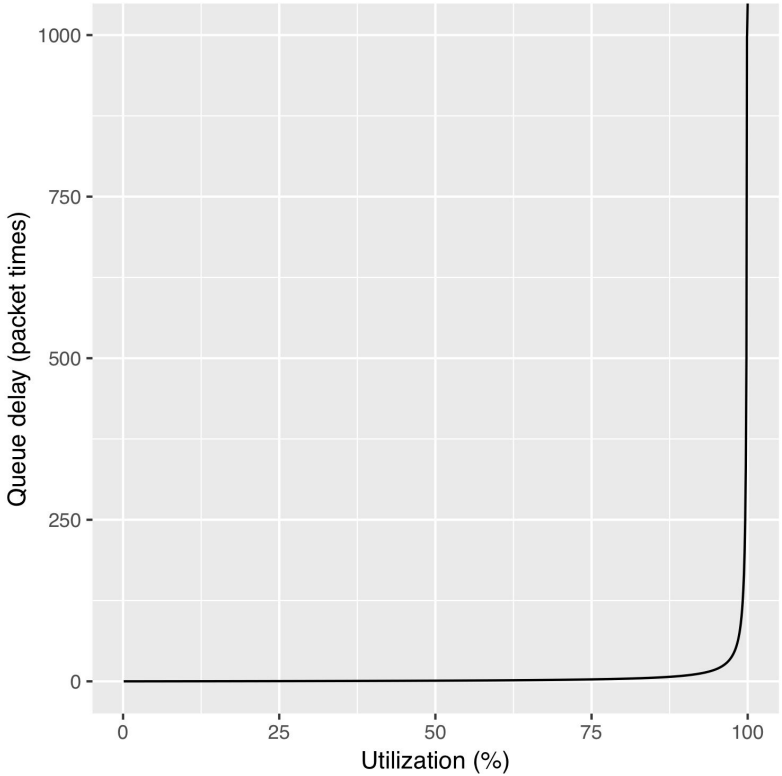
AFAP helped TCP/IP speed up 10,000x

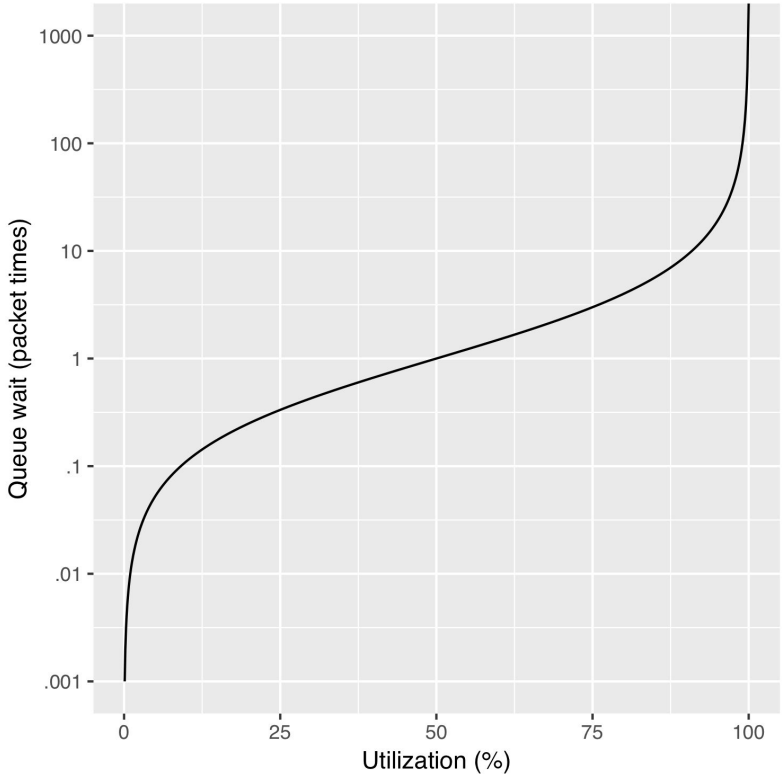


(25 years of Ethernet evolution)

… but AFAP makes bottleneck run at 100%

Queuing theory says this is fragile. E.g., for M/D/1:





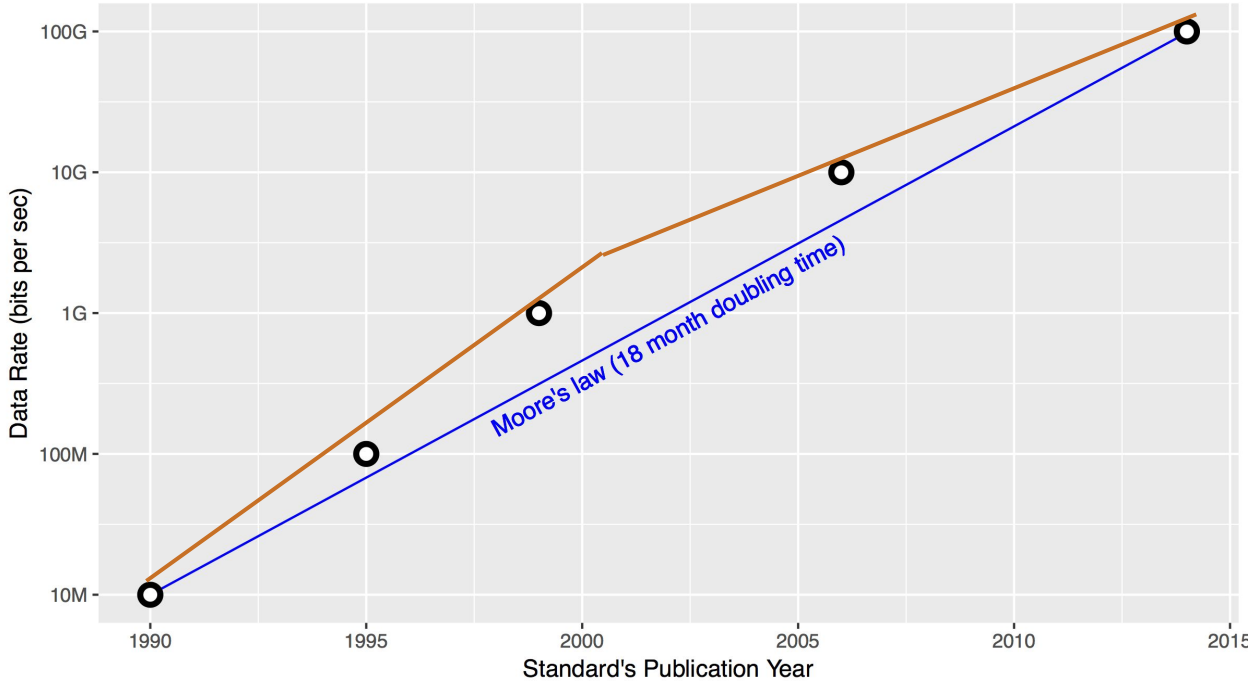
Delay/loss explosion isn’t an issue if:

- bandwidth delay products are small and/or
- there’s a fat-buffered router in front of every bottleneck and/or
- links from hosts to ToRs run slower than fabric

The first of these saved our ass until ~1995 then the second & third until ~2012.

Since then pain has been increasing.

After 2000, going faster got hard



Difficult to keep fabric switches 10x faster than server NICs

Pain points

- Jupiter (Sigcomm'15) – '*Jupiter Rising: A Decade of CLOS Topologies & Centralized Control in Google's Datacenter Network*'
- B4 (Sigcomm'13) – '*B4: Experience with a Globally Deployed Software Defined WAN*'

Pain relievers

- HULL (NSDI'12) – '*Less is more: trading a little bandwidth for ultra-low latency*'
- BwE (Sigcomm'15) – *'Flexible, Hierarchical Bandwidth Allocation for WAN'*
- FQ/pacing (IETF88'13) – *'TSO, fair queuing, pacing: three's a charm*'
- Timely (Sigcomm'15) – '*RTT-based congestion control for the datacenter*'
- BBR (CACM v60'17) – '*Congestion-based congestion control*'
- Carousel (Sigcomm'17) – '*Scalable traffic shaping at end hosts*'

AFAP isn't working for us now because it's *local* and our problems aren't.

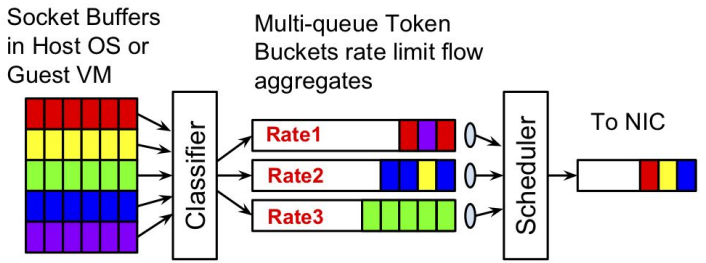
We need a mechanism that allows for more nuanced control of packet spacing on the wire.

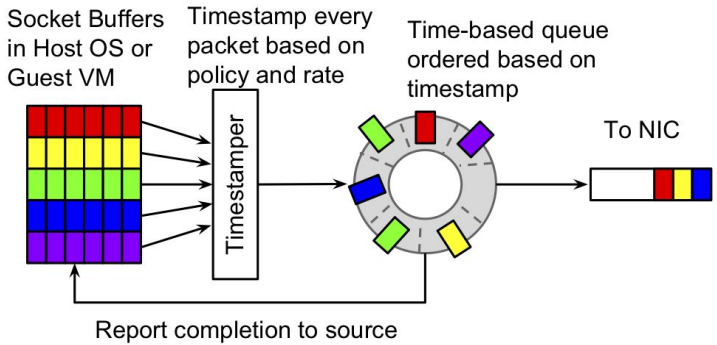
To enforce relationships between all outgoing packets, the enforcement *mechanism* needs to be just in front of (or in) the NIC.

Carousel is a great example of such a mechanism.

Carousel's core idea is to replace a complex of slow, brittle, concatenated queues with two simple pieces:

1. An *Earliest Departure Time (EDT)* timestamp in every skb
2. A timing-wheel scheduler replacing the queue in front of (or in) the NIC.



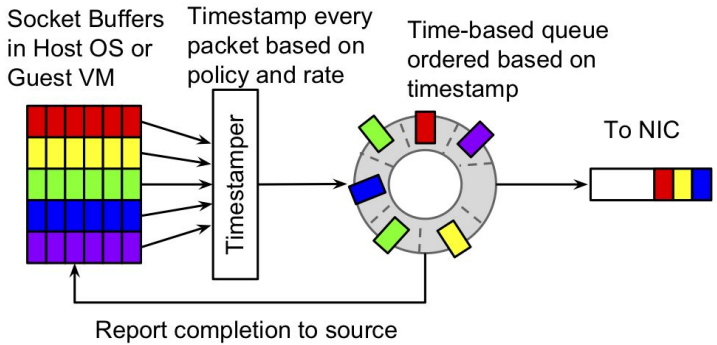


A timing wheel does what a queue does (and more) but is faster

Timing wheel insert & delete is O(1) like a queue but with a smaller multiplier:

- cache friendly (no pointer chains)
- RCU friendly (single slot to update)

Driver (or NIC) gets to choose ‘event horizon’ (wheel length) so can do BQL-like tuning for long enough to fill wire but short enough to not blow away caches.

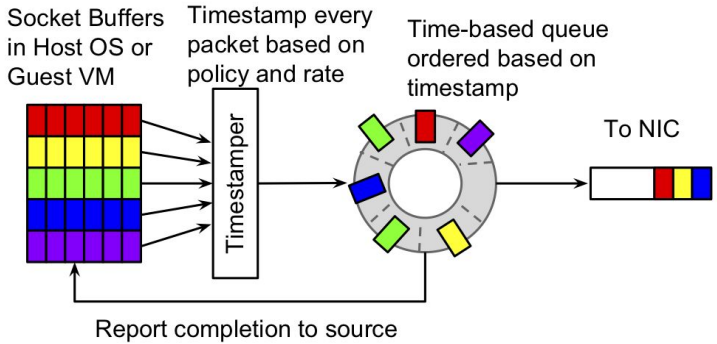


A timing wheel does what a queue does (and more) but is faster

Packets that would be sent after event horizon can get TSQ-like callback when they can be sent or get an *ETooFar*.

This replaces TSQ and fixes problem of many simultaneous writers generating huge queues.

It also puts hard bounds # of active output bytes, increasing probability of L3 cache hits for systems that can DMA from L3.

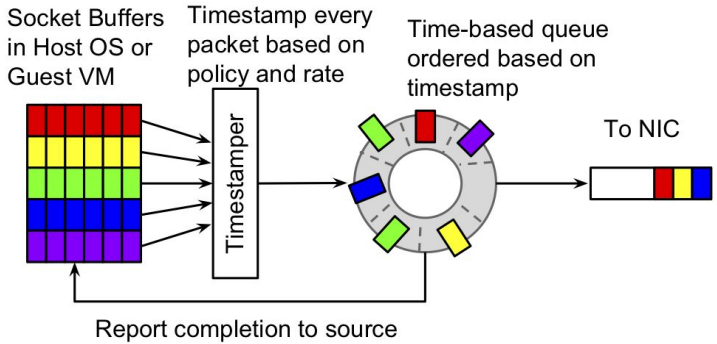


Qdiscs accomplish more with less

Qdiscs become purely computational – no more intermediate queues.

→ driver gets to see all packets in its event horizon so can easily do informed interrupt mitigation, lazy reclaim, (wifi) endpoint aggregation…

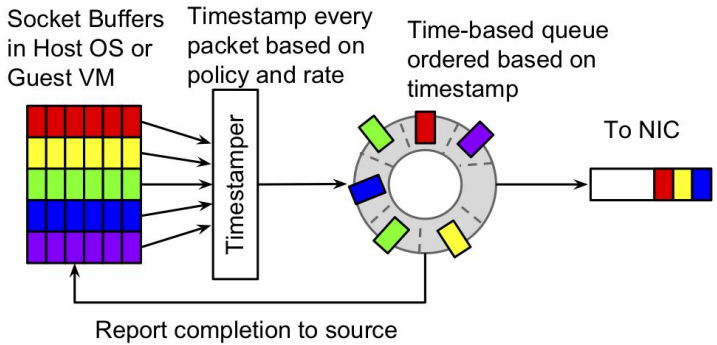
→ sender learns packet send time *on send()* and can adapt accordingly.



Qdiscs accomplish more with less

In essence, timing wheel is an in-memory representation of how packets will appear on wire. It can represent almost any causal scheduling policy.

(Policies like ‘Maximize Completion Rate’ are impossible to express with rates but easy with timestamps so we can finally make transactions ‘fair’ without stupidly slowing everything down.)



Questions?